

AI Destekli Mimari Karar Anlatımı

1. Başlangıç problemi

Side project generic CRUD yerine şu birleşimi seçti:

Digital Wallet + Double-entry Ledger + Fraud Detection + External Banking Integration

Amaç concurrency, idempotency, provider failure, accounting, session security, auditability ve reconciliation gibi gerçek financial-backend problemlerini göstermekti.

2. Neden modular monolith?

Wallet/Ledger/Transaction aynı atomik para hareketine katıldığı için başlangıçta microservice ayrıştırması distributed consistency maliyetini gereksiz erken getirdi. Domain modülleri logical olarak ayrıdır; deployment sınırı gerektiğinde sonra değerlendirilebilir.

3. Neden MSSQL financial source of truth?

ACID transaction, constraint, FK, unique key, explicit locking ve deterministic reconciliation query ihtiyacı vardır. Redis hızlı transient state için uygundur ama para var/yok kararının authority'si değildir.

4. Neden auth önce?

Wallet, BankAccount ve Transfer ownership kontrolü stabil customer/session identity ister. Bu yüzden registration -> OTP -> password -> login -> JWT -> refresh/session sırası money endpointlerinden önce kuruldu.

5. Neden JWT + server-side session?

JWT request authentication sağlar fakat tek başına immediate revoke sağlamaz. Minimal sid claim ve durable CustomerSession high-risk flow'da revoke/device/session kontrolü sağlar.

6. Neden Wallet ve BankAccount ayrı?

Wallet internal customer liability/balance state'tir. BankAccount external provider relationship'idir. Lifecycle ve source of truth farklı olduğu için aynı aggregate yapılmadı.

7. Neden FakeBank ayrı API?

External integration problemlerini gerçekten simüle etmek için HTTP boundary zorunlu tutuldu: timeout, 5xx, pending, provider-generated ID, idempotency ve polling. FinWallet provider DB'sine erişmez.

8. Neden durable state bank HTTP'den önce?

Bank-account opening önce internal BankAccount(Opening) kaydeder, SQL'i kapatır, sonra provider çağırır. Durable internal ID deterministic provider request key üretir; lost response duplicate external account'a dönüşmez.

9. Neden ledger transfer endpointinden önce?

Balance yalnız "şu an ne kadar var?" sorusunu cevaplar. Ledger "bu balance neden var?" sorusunun geçmişini sağlar. Bu nedenle money endpoint ledger invariant oluşmadan açılmaz.

10. FinancialTransaction neden LedgerJournal'dan ayrı?

FinancialTransaction business operation/lifecycle'dir; LedgerJournal accounting effect'tir. Failure/reversal/idempotency/resource identity ve reconciliation için ayrı kavramlar daha temizdir.

11. Neden explicit transfer store?

SqlWalletTransferPostingStore idempotency, iki wallet update'i, FinancialTransaction ve Ledger'ı tek MSSQL transaction'da sahiplenir. Bu bölgede locking/commit sırası correctness'in parçası olduğu için generic repository abstraction yerine explicit SQL tercih edildi.

12. Neden durable idempotency MSSQL'de?

Mobile timeout, gateway timeout, duplicate click veya retry aynı financial request'i tekrar gönderebilir. Final duplicate guarantee process cache/Redis'e bırakılmaz. Scope + CustomerId + Key + canonical payload hash kullanılır.

13. Neden fraud SQL money transaction'dan önce?

External provider HTTP uzun ve belirsizdir. Fraud önce değerlendirilir; yalnız final Allow sonrası kısa atomic financial SQL transaction açılır. Required fraud unavailable ise fail-closed.

14. Neden completed replay fraud'dan önce?

Dün completed olmuş aynı request bugün değişen fraud rule'ları yüzünden retry'da Deny olmamalıdır. Completed immutable result önce replay edilir.

15. Neden controller-based API?

Enterprise Web API convention, attributes, Swagger ve tutarlı action contractları için tüm HTTP servisleri controller standardında tutuldu; Minimal API ile iki convention karıştırılmadı.

16. ServiceResult neden yalnız HTTP contract?

ServiceResult<T> client response standardıdır. Domain/Application'ın HTTP envelope'a dependency alması business model ile transport katmanını gereksiz bağlardı.

17. YARP neden financial core'dan sonra?

Routing infrastructure yanlış para modelini düzeltmez. Auth, persistence, ledger ve transfer correctness kurulduktan sonra Gateway edge/platform boundary olarak eklendi.

18. Neden service-to-service de Gateway?

FinWallet provider adapterları /providers/* kullanır. İki credential vardır:

```
FinWallet -> Gateway    InternalServiceKey
Gateway -> Backend      DownstreamServiceKey
```

Bu Gateway'i yalnız client convention değil enforceable trust boundary yapar.

19. Neden Shared.Web?

Yedi Web API'ye ayrı ayrı Swagger/rate/CORS/header kodu kopyalamak configuration drift yarattı. FinWallet.Shared.Web yalnız host cross-cutting concern'leri merkezileştirir; business framework olmaz.

20. Neden bazı değerler config, bazıları değil?

Adres, timeout, pool, rate limit, body/header/connection limit, CORS, Swagger ve YARP destination operasyonel olduğu için config'tir. HS256 algorithm, PBKDF2 V1 migration semantics, Debit=Credit, financial decimal ve durable idempotency semantics correctness/security invariant olduğu için arbitrary runtime toggle değildir.

21. Neden Redis general cache olmadı?

Redis current use-case'te OTP/transient state için yeterlidir. Wallet balances veya final idempotency truth Redis'e taşınmadı; gereksiz cache invalidation ve correctness riski yaratılmadı.

22. Neden mock var ama yeterli sayılmıyor?

xUnit + Moq Application orchestration call davranışını test eder. SQL Serializable/range lock veya YARP route correctness gerçek infrastructure test ister.

23. Uygulama sırası

- scope/invariants
 - > architecture boundaries
 - > registration/auth/session
 - > communication/fraud providers
 - > wallet/ledger domain
 - > FakeBank + BankAccount
 - > persistence
 - > wallet APIs
 - > financial transaction/idempotency schema
 - > atomic transfer store
 - > fraud/session transfer orchestration
 - > YARP Gateway
 - > Shared.Web + Swagger/security
 - > config/performance tuning
 - > xUnit/Moq + CI tests
 - > bilingual/current documentation

24. Sıradaki doğal sıra

- 1 BankDeposit;
- 2 BankWithdrawal + cutoff;
- 3 integration/concurrency tests;
- 4 Outbox/Inbox + notification;
- 5 FraudEvents/manual review;
- 6 transaction history;
- 7 refund/reversal;
- 8 reconciliation;
- 9 OpenTelemetry/masked logging/operations.

25. Temel kural

Kolay endpoint'i, altındaki zor invariant hazır olmadan açma.

Transfer ledger/idempotency olmadan; bank flow durable state olmadan; provider retry idempotency olmadan expose edilmez.